XI. Java Features 1.x to Till Now

Posted by SmlCodes (http://www.smlcodes.com/author/satyakavetigmail-com/)

JDK Alpha and Beta (1995)

JDK 1.0 (23rd Jan, 1996) –

JDK 1.1 (19th Feb, 1997)

- AWT event model
- Inner classes
- JavaBeans
- JDBC
- RMI,Reflection
- JIT (Just In Time) compiler for Windows

J2SE 1.2 (8th Dec, 1998) – Playground

- **strictfp** keyword
- Swing graphical API
- Sun's JVM was equipped with a JIT compiler for the first time
- Java plug-in
- **Collections framework**

J2SE 1.3 (8th May, 2000) – Kestrel

- HotSpot JVM
- Java Naming and Directory Interface (JNDI)
- Java Platform Debugger Architecture (JPDA)
- JavaSound
- Synthetic proxy classes

J2SE 1.4 (6th Feb, 2002) – Merlin

- **assert keyword**
- **Regular expressions**
- Exception chaining
- Internet Protocol version 6 (IPv6) support
- New I/O; NIO
- **Logging API**
- Image I/O API
- Integrated XML parser and XSLT processor (JAXP)
- Integrated security and cryptography extensions (JCE, JSSE, JAAS)
- Java Web Start
- Preferences API (java.util.prefs)

J2SE 5.0 (30th Sep, 2004) – Tiger

- **Generics**
- **Annotations**
- **Autoboxing/unboxing**
- **Enumerations**
- **Varargs**
- **Enhanced for each loop**
- **Static imports**
- **New concurrency utilities in java.util.concurrent**
- **Scanner class for parsing data from various input streams and buffers.**

Java SE 6 (11th Dec, 2006) – Mustang

- Scripting Language Support
- Performance improvements
- JAX-WS
- JDBC 4.0
- Java Compiler API
- JAXB 2.0 and StAX parser
- Pluggable annotations
- New GC algorithms

Java SE 7 (28th July, 2011) – Dolphin

- JVM support for dynamic languages
- Compressed 64-bit pointers
- **Strings in switch**
- Automatic resource management in try-statement
- The diamond operator
- Simplified varargs method declaration
- Binary integer literals
- Underscores in numeric literals
- Improved exception handling
- ForkJoin Framework
- NIO 2.0 having support for multiple file systems, file metadata and symbolic links
- WatchService
- Timsort is used to sort collections and arrays of objects instead of merge sort
- APIs for the graphics features
- Support for new network protocols, including SCTP and Sockets Direct Protocol

Java SE 8 (18th March, 2014) – Code name culture dropped

- **Lambda expression support in APIs**
- Functional interface and default methods
- Optionals
- Nashorn – JavaScript runtime which allows developers to embed JavaScript code within applications
- Annotation on Java Types
- Unsigned Integer Arithmetic
- Repeating annotations
- New Date and Time API
- Statically-linked JNI libraries
- Launch JavaFX applications from jar files
- Remove the permanent generation from GC

Java SE 9 Expected: September 22, 2016

- Support for multi-gigabyte heaps
- Better native code integration
- Self-tuning JVM
- Java Module System

- Money and Currency API
- jshell: The Java Shell
- Automatic parallelization

1.1 Assert keyword

Assert keyword is used to check the given statement is **TRUE or FALSE**

There are two types of using assert in our program

1. **assert(boolean** expression); //Simple assert
2. **assert(boolean** expression1) : (**anytype** expression2); //Agumented assert

Assertion is disabled by default. To enable we have to use **java –ea classname** or **-enableassertions**

The main advantage of assert is for **DEBUGGING**. If we write s.o.p's for debugging after completion of code we have to munually remove the s.o.p's. But if we use assertions for debugging after completion of code we don't need to remove the code, just DISABLING assertion is enough

11.1.1 Simple assert

```
assert ( boolean expression);
```

- Here the Expression Should be **Boolean**
- If expression is TRUE it wont return anything,
- Otherwise it will throws **Runtime Exception** : **lang.AssertionError: Not valid**

```
1 public class AssertDemo {
2
3 public static void main(String[] args) {
4 int i = 100;
5 assert (i>10);
6 System.out.println(i); //100
7 }
8
9 }
```

If we give **int** i = 10; it will throws **java.lang.AssertionError: Not valid**

11.1.2 Agumented assert

```
assert ( boolean expression1) : ( anytype expression2); //Agumented assert
```

Expression2 –> is used to Disply some message along with Error Message

- Here the 1st Expression Should be **Boolean** type, 2nd Expression can be **Any type**
- If expression is TRUE it wont return anything,
- Otherwise it will throws **Runtime Exception : lang.AssertionError: expression2**

```
1 public class AssertDemo {
2
3 public static void main(String[] args) {
4 int i = 1;
5 assert (i > 10) : "This is Anytype";
6 System.out.println(i);
7 }
8
9 }
```

```
1 Exception in thread "main" java.lang.AssertionError: This is Anytype
2 at features.AssertDemo.main(AssertDemo.java:6)
```

To ENABLE assertions we have to use **java -ea classname or -enableassertions**

To DISABLE assertions we have to use **java -da classname or -disableassertions**

To ENABLE assertions in **ECLIPSE** Rightclickon File --> Run As -->Run Configurations -->Apply-->Save

11.2 Enhanced for each loop

- For-each loop introduced in **Java 1.5 version as an enhancement of traditional for-loop**
- Main advantage is we **don't need to write extra code** traverse over array / collections
- Mainly used for traverse on **Array Elements & Collection Elements**

for (**Datatype** temp_variable : **Array/Collection** Variable){}

11.2.1 for-each loop on Array Elements

```
1 public class Foreach {
2 public static void main(String[] args) {
3 int i[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
4 for (int b : i) {
5 System.out.print(b+", ");
6 }
7 }
8 }
```

Output : 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,

11.2.2 for-each loop on Collection Elements

```
1  public class Foreach {
2
3  public static void main(String[] args) {
4  ArrayList<String> l = new ArrayList<String>();
5  l.add("A");
6  l.add("B");
7  l.add("C");
8  l.add("D");
9  l.add("E");
10 for (String s : l) {
11 System.out.print(s + ",");
12 }
13 }
14 }
```

Output : A,, B, C, D, E

11.3 Var-args

Some times we don't know no.of arguments used in our method implementation.var…args are introduced in 1.5v to solve these type of situations

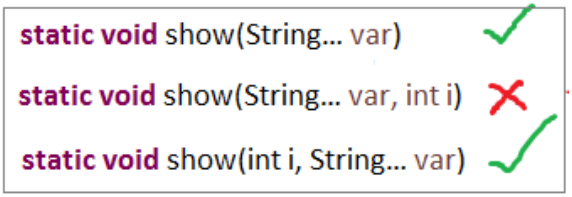
```
static   returntype  methodname( Datatype  … variblename)
```

```
1  public class Varargs {
2      static void show(String… var) {
3          System.out.println("Show() called");
4      }
5      public static void main(String[] args) {
6          Varargs v = new Varargs();
7          v.show();
8          v.show("A");
9          v.show("A", "B", "C");
10     }
11 }
```

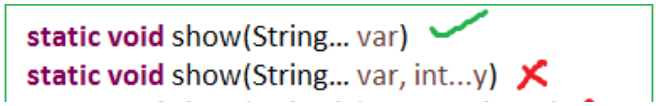
```
1 Show() called
2 Show() called
3 Show() called
```

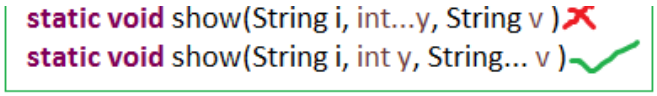
Rules for using Var-args

- **Var-args must be as the last argument in method signature**



- **Only one Var-arg is allowed per a Method**





11.4 Static imports

From 1.5 version onwords we can access any static member of a class directly. There is no need to qualify it by the class name.

```
import static java.lang.System.*;
```

```
1 import static java.lang.System.*;
2 public class StaticImport {
3 public static void main(String[] args) {
4 out.println("Static Import");
5 }
6 }
```

11.5 Enums

Enum in java is a data type that contains fixed set of constants.

- enums improves **type safety**
- easily used in **switch**
- enum can be traversed
- enum class is a just like normal class, we can write FIELDS, Constrcutors, Methods in enum class
- enum **CANNOT extend any class** because it internally extends Enum class
- enum may **implement many interfaces**

ALL fields in Enums by Default **PUBLIC STATIC FINAL**

1.Simple ENUM Example

```
1  enum Days {
2      MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
3  }
4
5  public class EnumDemo {
6      public static void main(String[] args) {
7          for (Days s : Days.values()) {
8              System.out.print(s + ",");
9          }
10     }
11 }
```

Output : ***MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY,***

2.Enum with Values

To write Enum with values we have to follow below steps

- Enum values must be placed inside ( ) –> ***MONDAY(1)***
- Take public instance variable to Store enum value –> ***public int Enum_Value;***
- Write **private constructor** which can take enum value as argument –> **private Days(int Enu_Val)**

```
1  enum Days {
2  MONDAY(1), TUESDAY(2), WEDNESDAY(3), THURSDAY(4), FRIDAY(5), SATURDAY(6), SUNDAY(7);
3      public int Enum_Value;
4      private Days(int Enum_Value) {
5          this.Enum_Value = Enum_Value;
6      }
7  }
8  public class EnumDemo {
9      public static void main(String[] args) {
10         for (Days s : Days.values()) {
11             System.out.println(s + ":"+s.Enum_Value);
12         }
13     }
14 }
```

Output: ***MONDAY :1, TUESDAY:2, WEDNESDAY:3, THURSDAY :4, FRIDAY:5, SATURDAY:6 SUNDAY:7***

11.6 Annotations

Annotations in java are used to **provide additional information**, so it is an **alternative option for XML and java marker interfaces**

Java @Annotation is a represents **metadata**. It can be attached at the top of class, interface, methods or fields to indicate some additional information which can be used by java compiler and JVM

1.Built-In Annotations in Java

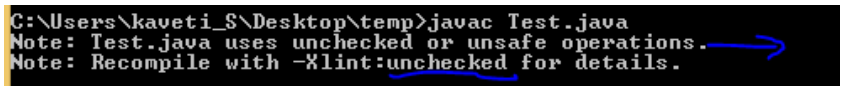
i.@Override

It will indicates that the Subclass is overriding Parent class method in its class

```
1 public class Test {
2
3 @Override
4 public String toString() {
5 return "toString() Override";
6 }
7
8 }
```

ii.@SuppressWarnings

If we use **ArrayList** directly without using generic collection like **ArrayList<String>,** at compile time it shows warnings like



To avoid those type of warnings we can use **@SuppressWarnings.**It will Supress/reduces/Avoids those type of warings. In above it says Recomple with **unchecked** .So by adding @SuppressWarnings("unchecked") it wont shows any warings at compile time

```
1  public class Test {
2      @SuppressWarnings("unchecked")
3      public static void main(String args[]) {
4          ArrayList list = new ArrayList();
5          list.add("A");
6          list.add("B");
7          list.add("C");
8
9          for (Object obj : list)
10             System.out.println(obj);
11     }
12 }
```

iii.@Deprecated

@Deprecated annotation informs user that it may be removed in the future versions. So, it is better not to use such methods.

2.Custom Annotations

We can create our Own Annonations by following below Steps

- Add **@interface** before annotation name
- Every **method should return any one primitive** value (String,Int,etc)
- We can give default value to methods by using **default val**;
- Method should not throws any exceptions

We have 3 types of Annotations

- **Marker Annotation**
- **Single-valued Annotation**
- **Multi-valued Annotation**

1. **Marker Annotation:** An annotation that has no method, is called marker annotation

```
@interface  MyAnnotation{}
```

2. **Single-Valued Annotation:** An annotation that has one method is called single-value annotation

```
1 @interface MyAnnotation{
2
3 int value() default 0;
4
5 }
```

3. **Multi-Valued Annotation:** An annotation that has more than one method

```
1 @interface MyAnnotation{
2
3 int value1() default 1;
4 String value2() default "";
5 String value3() default "xyz";
6
7 }
```

For every annontation we can place **@Target, @Retention annotations** to specify where we are going to use our custom annotation

@Target annotation is used to specify at which type, the annotation is used

Element Types	Where the annotation can be applied
ElementType.TYPE	class, interface or enumeration
ElementType.FIELD	fields
ElementType.METHOD	methods
ElementType.CONSTRUCTOR	constructors
ElementType.LOCAL_VARIABLE	local variables
ElementType.ANNOTATION_TYPE	annotation type
ElementType.PARAMETER	parameter

@Retention annotation is used to specify to what level annotation will be available.

RetentionPolicy	Availability
RetentionPolicy.SOURCE	Refers to the source code, discarded during compilation. It will not be available in the compiled class.
RetentionPolicy.CLASS	Refers to the .class file, available to java compiler but not to JVM. It is included in the class file.
RetentionPolicy.RUNTIME	refers to the runtime, available to java compiler and JVM

Simple Custom annotation Example

```
1 @Retention(RetentionPolicy.RUNTIME)
2
3 @Target(ElementType.METHOD)
4
5 @interface MyAnnotation{
6 int value();
7 }
```

11.7 Generics

Generics are introduced in Java 1.5 Version to solve **Type-safity** & **Type-casting** problems

```
1  public class Student {
2
3  public static void main(String[] args) {
4  ArrayList l = new ArrayList(); //1
5  l.add("Satya");
6
7  String s = (String) l.get(0); //2
8  System.out.println(s);
9  }
10
11 }
```

- At **line 1.** ArrayList is **NOT generic** type – so we can add any type of elements results **Type-Safity**
- At **line 2.** It returns added String data as Object. So manullay we have to **Type-cast to String**

To resolve above problems Genrics are indrodued

```
1  public class Student {
2
3  public static void main(String[] args) {
4  ArrayList<String> l = new ArrayList<String>(); // 1
5  l.add("Satya");
6  String s = l.get(0); // 2
7  System.out.println(s);
8  }
9
10 }
```

Generics solves ClassCastException, beacause TYPE-CHECKING done at COMPILE-TIME itself

Generics can be used in following areas

- *Class level*
- *Interface level*
- *Constructor level.*
- *Method level*

Sample Generic class

```
1  class SmlGen<T> {
2      T obj;
3
4      void add(T obj) {
5          this.obj = obj;
6      }
7
8      T get() {
9          return obj;
10     }
11 }
```

The **<T>** indicates that it can refer to any type (like String, Integer, and Student etc.). The type you specify for the class, will be used to store and retrieve the data

Type Parameter<T> Naming Conventions

In above <T> refers any type. Similary we have to follow below naming convensions to where to use which Naming conevsions. It improves readability. These are NOT compulsory but recommended to use

1. **T – Type – Any Type, can be use any level (Class, Interface, Methods...)**
2. **N – Number – Indicates it allows Number Types (int,long,float etec)**
3. **E – Element – Exclusivly used in Collection (ArrayList<E>)**
4. **K – Key – Map KEY area**
5. **V – Value – Map Value area**
6. **S,U,V etc. – 2nd, 3rd, 4th types**

1.Genrics at Class /Method /Constrcutor level

A generic class is defined with the following format:

```
class name<T1, T2, ..., Tn> {
---
 }
---------------------------------

class SmlGen<T> {             //1.Class Level
    T obj;
    public SmlGen() {
            }

    public SmlGen(T obj) {      //2.Constrcutor Level
        this.obj = obj;
    }

    void add(T obj) {
        this.obj = obj;
    }

    T get() {               //3.Method Level
        return obj;
    }
}

public class Student {
    public static void main(String[] args) {
        SmlGen<String> s = new SmlGen<String>();
        s.add("Satya");
        System.out.println(s.get());
    }
}
```

2.Genrics at Interface level

A generic interface is same as generic class.

```
public interface List<T> extends Collection<T> {
...
}
```

3.Wildcard in Generics

? Operator is used to represents wilcards in generics. We have to types of Wildcards

1. **Unbounded wildcards**
2. **Bounded wildcards**

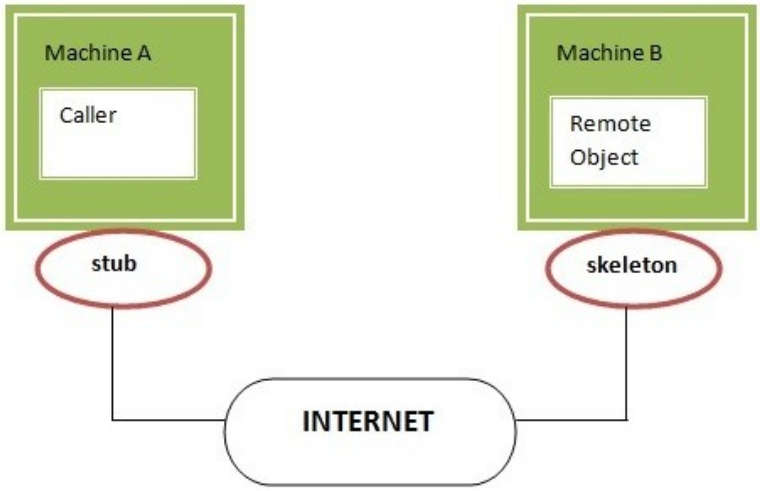
1.Unbounded wildcard looks like <?> – means the generic can be any type.it is not bounded with anytype.

2.Bounded wildcards

- <? extends T> and <? super T> are examples of bounded wildcards
- <? extends T> : means it can accept the **Child class Objects of the type<T>**
- <? super T> : means it can accept the **Partent class Objects of the type<T>**

11.8 RMI

RMI used to invoke methods which is running on one JVM from another JVM



1. **Stub**

The stub is an object, acts as a gateway for the client side.if we invokes method on the stub object, it does the following tasks:

- It **initiates a connection with remote Virtual Machine (JVM),**
- It **writes and sends** (marshals) the parameters to the remote Virtual Machine (JVM),
- It waits for the result
- It **reads** (unmarshals) the **return value or exception**, and

2. **Skeleton:** The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

- It reads the parameter for the remote method
- It invokes the method on the actual remote object, and
- It writes and transmits (marshals) the result to the caller.

Steps to Create Skeleton / Client

1. Create an **Interface by implementing Remote** interface with methods you want to share
2. Create a **Class** by exctending UnicastRemoteObject & also implement above methods
3. Create a **class to Share the Remote Class Object** over the Network

1.Interface by implementing Remote

```
1 public interface RemoteInterface extends Remote {
2
3 public String show(String name);
4
5 }
```

2.Class by exctending UnicastRemoteObject

```
1  public class RemoteClass extends UnicastRemoteObject implements RemoteInterface {
2      protected RemoteClass() throws RemoteException {
3          super();
4      }
5      @Override
6      public String show(String name) {
7          // TODO Auto-generated method stub
8          return "Your Name Is : " + name;
9      }
10 }
```

3. Create a class to Share the Remote Class Object over the Network

```
1 public class RemoteObject {
2
3 public static void main(String[] args) throws RemoteException, MalformedURLException, AlreadyBo
4 RemoteInterface obj = new RemoteClass();
5 Naming.rebind("obj",  obj);
6 }
7
8 }
```

4.stub class – Client

```
1 public class Client {
2
3 public static void main(String args[]) throws Exception
4 RemoteInterface st=(RemoteInterface) Naming.lookup("rmi://"+args[0]+ "/obj");
5 System.out.println(st.show("Satya"));
6 }
7 }
```

11.9 RegExp

Java.util.regex or Regular Expression is an API to **define pattern for searching or manipulating strings.** It is widely used to define constraint on strings such as **password and email validation.**

It provides following classes are widely used in java regular expression.

- Pattern class -it represents the Complied pattern
- Matcher class -used for performing matching operations on complied pattern
- PatternSyntaxException – checks syntax error in a regular expression pattern.

1. **Pattern class** it represents the Complied pattern

Method	Description
static Pattern **compile**(String regex)	Compiles the given regex and return the instance of pattern.
Matcher **matcher**(CharSequence input)	Retuns charsequence to be comapir with patten
static boolean **matches(String regex, String CharSequence)**	Direcly we can compare Expression with Sequence
String pattern()	returns the regex pattern.

2. **Matcher class** -used for performing matching operations on complied pattern

Method	Description
boolean **matches()**	Test whether the regular expression matches the pattern.
boolean find()	Finds the next expression that matches the pattern.
boolean find(int start)	Finds the next expression that matches the pattern from the given start number.
String group()	Returns the matched subsequence.
int start()	Returns the starting index of the matched subsequence.
int end()	Returns the ending index of the matched subsequence.
int groupCount()	Returns the total number of the matched subsequence.

1.
```
1  public class REDemo {
2     public static void main(String[] args) {
3      Pattern p = Pattern.compile(".a");// only 2 char end with a
4      Matcher m = p.matcher("sa");
5      boolean b1 = m.matches();
6      System.out.println(b1);//TRUE
7
8      boolean b2 = Pattern.matches("s.", "sa"); //only 2 char Start with s
9      System.out.println(b2); //TRUE
10     }
11 }
```

Regex Character classes

Character Class	Description
[abc]	a, b, or c (simple class)
[^abc]	Any character except a, b, or c (negation)
[a-zA-Z]	a through z or A through Z, inclusive (range)
[a-d[m-p]]	a through d, or m through p: [a-dm-p] (union)
[a-z&&[def]]	d, e, or f (intersection)
[a-z&&[^bc]]	a through z, except for b and c: [ad-z] (subtraction)
[a-z&&[^m-p]]	a through z, and not m through p: [a-lq-z](subtraction)

2. **Regex Quantifiers:**The quantifiers specify the number of occurrences of a character.

Regex	Description
X?	X occurs once or not at all
X+	X occurs once or more times
X*	X occurs zero or more times
X{n}	X occurs n times only
X{n,}	X occurs n or more times
X{y,z}	X occurs at least y times but less than z times

3. **Regex Metacharacters**: The regular expression metacharacters work as a short codes.

Regex	Description
. (dot)	Any character (may or may not match terminator)
\d	Any digits, short of [0-9]
\D	Any non-digit, short for [^0-9]
\s	Any whitespace character, short for [\t\n\x0B\f\r]
\S	Any non-whitespace character, short for [^\s]
\w	Any word character, short for [a-zA-Z_0-9]
\W	Any non-word character, short for [^\w]
\b	A word boundary
\B	A non word boundary

```
1  public class REDemo {
2      public static void main(String[] args) {
3  S.o.p("1.Regex Character classes\n----------------");
4  S.o.p(Pattern.matches("[amn]", "abcd"));//false (not a or m or n)
5  S.o.p(Pattern.matches("[amn]", "a"));//true (among a or m or n)  S.o.p(Pattern.matches("[amn]"
6
7  S.o.p("\n2.Regex Quantifiers\n----------------");
8  S.o.p("? quantifier ....");
9  S.o.p(Pattern.matches("[amn]?", "a"));//true (a or m or n comes one time)
10 S.o.p(Pattern.matches("[amn]?", "aaa"));//false (a comes more than one time)
11 S.o.p(Pattern.matches("[amn]?", "aammmnn"));//false (a m and n comes more than one time)
12
13 S.o.p("+ quantifier ....");
14 S.o.p(Pattern.matches("[amn]+", "a"));//true (a or m or n once or more times)
15 S.o.p(Pattern.matches("[amn]+", "aaa"));//true (a comes more than one time)
16
17
18 S.o.p("\n3.Regex Metacharacters\n----------------\n");
19 S.o.p(Pattern.matches("\\d", "abc"));//false (non-digit)
20 S.o.p(Pattern.matches("\\d", "1"));//true (digit and comes once)
21 S.o.p(Pattern.matches("\\d", "4443"));//false (digit but comes more than 1)
22     }
23 }
```

11.10 Logging API

In common we use **System.out.println ()** statements for DEBUGGING. But these are printed at console and they will lost after closing the Console.so these results are not savable

To overcome these problems apache released **Log4j.** With Log4j we can store the flow details of our Java/J2EE in a file or databases

We have mainly 3 components to work with Log4j

- **Logger class -for printing LOG messages**
- **Appender interface -to store messages in Files/Databases**
- **Layout -which Fomate the message should Save(HTML,Text,etc)**

1. **Logger class**

- Logger is a class, in org.apache.log4j.*
- We need to create **Logger object one per java class**,it will enables Log4j in our java class
- Logger methods are used to generate log statements in a java class instead of sopls
- So in order to get an object of Logger class, we need to call a **static factory method**

static Logger log = Logger.getLogger(YourClassName.class.getName())

We have following methods to print debugging statements on Logger

1. **debug (" ")**
2. **info ("")**
3. **warn ("")**
4. **error ("")**

5. **fatal (“”)**

Here **human identification purpose names are different**, all 5 methods will print one text message only.

Priority Order: debug < info < warn < error < fatal

2. **Appender interface**

Appender job is to write the messages into the **external file or database or SMTP**In log4j we have different Appender implementation classes

- **ConsoleAppende**r [Writing into console]
- **FileAppender** [writing into a file]
- **JDBCAppender** [For Databases]
- **SMTPAppender** [sent logs via Mails]
- **SocketAppender** [For remote storage]

3. **Layout**

This component specifies the format in which the log **statements are written into the destination** by the appender

- **SimpleLayout**
- **PatternLayout**
- **HTMLLayout**
- **XMLLayout**

Simple Hello world

```
1  public class LogDemo {
2  public static void main(String[] args) {
3      Logger logger = Logger.getLogger(LogDemo.class.getName());
4      Layout layout = new SimpleLayout();
5      Appender a = new ConsoleAppender(layout);
6      logger.addAppender(a);
7
8      logger.debug("Debug Message");
9      logger.info("Info Message");
10     logger.warn("Warning Message");
11     logger.error("Error Message");
12     logger.fatal("Fatal Message");
13 }
14 }
```

In above Example we used **Layout, Appenders** programmatically which is NOT RECOMMENDED.we have to use **log4j.properties** to configure those.

Log4j.properties Structure

```
log4j.rootLogger=DEBUG, CONSOLE, LOGFILE
log4j.appender.CONSOLE=
log4j.appender.CONSOLE.layout=
log4j.appender.CONSOLE.layout.ConversionPattern=
log4j.appender.LOGFILE=
log4j.appender.LOGFILE.File=
log4j.appender.LOGFILE.MaxFileSize=
log4j.appender.LOGFILE.layout=
log4j.appender.LOGFILE.layout.ConversionPattern=
```

If we use .properties file, we **no need to import any related classes** into our java class

if we wrote **log4j.rootLogger = WARN,abc** then it will prints the messages in l.warn(), l.error(), l.fatal() and ignores l.debug(), l.info(). Means >Warn level only it prints

Make sure LOG file should be placed in /src Folder

Example programe to Store LOG's in a FILE

```
public class LogDemo {
    static Logger logger = Logger.getLogger(LogDemo.class.getName());
    public static void main(String[] args) {
        logger.debug("Debug Message");
        logger.info("Info Message");
        logger.warn("Warning Message");
        logger.error("Error Message");
        logger.fatal("Fatal Message");
    }
}
```

Log4j.properties

```
log4j.rootLogger = DEBUG,abc
log4j.appender.abc = org.apache.log4j.FileAppender
log4j.appender.abc.file = logfile.log
log4j.appender.abc.layout = org.apache.log4j.SimpleLayout
```

logfile.log

```
DEBUG - Debug Message

INFO - Info Message

WARN - Warning Message

ERROR - Error Message

FATAL - Fatal Message
```

The above example only saves log's to file. You can't see logs on console .if want both use below. Use same java program, but change log4j.properties file

log4j.properties in Real world Applications

```
log4j.rootLogger=DEBUG,CONSOLE,LOGFILE
log4j.appender.CONSOLE=org.apache.log4j.ConsoleAppender
log4j.appender.CONSOLE.layout=org.apache.log4j.PatternLayout
log4j.appender.CONSOLE.layout.ConversionPattern=%-4r [%t] %-5p %c %x - %m%n
log4j.appender.LOGFILE=org.apache.log4j.RollingFileAppender
log4j.appender.LOGFILE.File=logfile.log
log4j.appender.LOGFILE.MaxFileSize=1kb
log4j.appender.LOGFILE.layout=org.apache.log4j.PatternLayout
log4j.appender.LOGFILE.layout.ConversionPattern=[%t] %-5p %c %d{dd/MM/yyyy HH:mm:ss} - %m%n
```

```
[main] DEBUG log.LogDemo 15/09/2016 19:23:38 â?? Debug Message
[main] INFO  log.LogDemo 15/09/2016 19:23:38 â?? Info Message
[main] WARN  log.LogDemo 15/09/2016 19:23:38 â?? Warning Message
[main] ERROR log.LogDemo 15/09/2016 19:23:38 â?? Error Message
[main] FATAL log.LogDemo 15/09/2016 19:23:38 â?? Fatal Message
```

Share with Friend

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/?share=twitter&nb=1)

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/?share=facebook&nb=1)

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/?share=google-plus-1&nb=1)

(whatsapp://send?text=XI.%20Java%20Features%201.x%20to%20Till%20Now http%3A%2F%2Fwww.smlcodes.com%2Fcore-java-complete%2Fxi-features-1-x-till-now%2F)

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/?share=pinterest&nb=1)

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/?share=linkedin&nb=1)

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/?share=skype&nb=1)

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/?share=tumblr&nb=1)

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/?share=email&nb=1)

(http://www.smlcodes.com/core-java-complete/xi-features-1-x-till-now/#print)

Like this:

Like

Be the first to like this.

Related

I.Introduction to Java -Core Java Complete Tutorial (http://www.smlcodes.com/java/introduction-java-core-java-complete-tutorial/)
September 17, 2016
In "core-java-complete"

IV. Java Exception Handling (http://www.smlcodes.com/core-java-complete/iv-java-exception-handling/)
September 24, 2016
In "core-java-complete"

So Called CORE JAVA - Satya Kaveti's eBook Download (http://www.smlcodes.com/onepage-tutorial/socalled-core-java-satya-kavetis-ebook-download/)
September 25, 2016
In "Books & Downloads"

core-java-complete (http://www.smlcodes.com/category/core-java-complete/), Java (http://www.smlcodes.com/category/java/)
corejava (http://www.smlcodes.com/tag/corejava/)

← X. Java Networking (java.net. *) (http://www.smlcodes.com/core-java-complete/x-java-networking-java-net/)

XII Design Patterns & EJB Just Introduction → (http://www.smlcodes.com/core-java-complete/xii-design-patterns-ejb-just-introduction/)

Leave a Reply

Enter your comment here...

Search …

Recent Posts

Archives

Categories

Books & Downloads (http://www.smlcodes.com/category/books-downloads/)

Carrier Tips (http://www.smlcodes.com/category/carrier-tips/)

core-java-complete (http://www.smlcodes.com/category/core-java-complete/)

Databases (http://www.smlcodes.com/category/databases/)

DevOps (http://www.smlcodes.com/category/devops/)

Errors (http://www.smlcodes.com/category/errors/)

HTML (http://www.smlcodes.com/category/html/)

jar (http://www.smlcodes.com/category/jar/)

Java (http://www.smlcodes.com/category/java/)

JSON (http://www.smlcodes.com/category/json/)

jsoup (http://www.smlcodes.com/category/jsoup/)

Onepage Tutorial (http://www.smlcodes.com/category/onepage-tutorial/)

PHP (http://www.smlcodes.com/category/php/)

Servers (http://www.smlcodes.com/category/servers/)

Tools (http://www.smlcodes.com/category/tools/)

XML (http://www.smlcodes.com/category/xml/)

© 2016 smlcodes.com